Árboles y Grafos, 2026-1

Para entregar el domingo 8 de febrero de 2026
A las 23:59 en la arena de programación

Instrucciones para la entrega

- Para esta tarea y todas las tareas futuras en la arena de programación, la entrega de soluciones es *individual*. Por favor escriba claramente su nombre, código de estudiante y sección en cada archivo de código (a modo de comentario). Adicionalmente, cite cualquier fuente de información que utilizó. Los códigos fuente que suba a la arena de programación deben der de su completa autoría y no está permitido resolver los ejercicios con la ayuda de internet, herramientas de ingeligencia articial ni otras personas.
- En cada problema debe leer los datos de entrada de la forma en la que se indica en el enunciado y debe imprimir los resultados con el formato allí indicado. No debe agregar mensajes ni agregar o eliminar datos en el proceso de lectura. La omisión de esta indicación puede generar que su programa no sea aceptado en la arena de programación.
- Puede resolver los ejercicios en C/C++ y Python. Sin embargo, deben haber por los menos soluciones a dos problemas en cada lenguaje.
- Debe enviar sus soluciones a través de la arena. Antes de subir sus soluciones asegurese de realizar pruebas con los casos de pruebas proporcionados para verificar que el programa finalice y no se quede en un ciclo infinito.
- El primer criterío de evaluación será la aceptación del problema en la arena cumpliendo los requisitos indicados en los enunciados de los ejercicios y en este documento. El segundo criterio de evaluación será la complejidad computacional de la solución. Además, se tendrán en cuenta aspectos de estilo como no usar `break` ni `continue` y que las funciones deben tener únicamente una instrucción `return` que debe estar en la última línea.

Problemas prácticos

Hay cinco problemas prácticos cuyos enunciados aparecen a partir de la siguiente página. Debe resolver 4 de los 5 problemas. El problema restante se calificará con **0.5 décimas de bonificación**. En caso de que la nota total de la tarea supere 5.0 el excedente será puesto en otra tarea del curso.

A - Problem A

Source file name: `accordian.cpp, accordian.py`
Time limit: c++: x seconds, python: x seconds

You are to simulate the playing of games of "Accordian" patience, the rules for which are as follows:

Deal cards one by one in a row from left to right, not overlapping. Whenever the card matches its immediate neighbour on the left, or matches the third card to the left, it may be moved onto that card. Cards match if they are of the same suit or same rank. After making a move, look to see if it has made additional moves possible. Only the top card of each pile may be moved at any given time. Gaps between piles should be closed up as soon as they appear by moving all piles on the right of the gap one position to the left. Deal out the whole pack, combining cards towards the left whenever possible. The game is won if the pack is reduced to a single pile.

Situations can arise where more than one play is possible. Where two cards may be moved, you should adopt the strategy of always moving the leftmost card possible. Where a card may be moved either one position to the left or three positions to the left, move it three positions.

Input

Input data to the program specifies the order in which cards are dealt from the pack. The input contains pairs of lines, each line containing 26 cards separated by single space characters. The final line of the input file contains a `'#'` as its first character. Cards are represented as a two character code. The first character is the face-value (`A`=Ace, 2–9, `T`=10, `J`=Jack, `Q`=Queen, `K`=King) and the second character is the suit (`C`=Clubs, `D`=Diamonds, `H`=Hearts, `S`=Spades).

The input must be read from standard input.

Output

One line of output must be produced for each pair of lines (that between them describe a pack of 52 cards) in the input. Each line of output shows the number of cards in each of the piles remaining after playing "Accordian patience" with the pack of cards as described by the corresponding pairs of input lines.

The output must be written to standard output.

Sample Input

```
QD AD 8H 5S 3H 5H TC 4D JH KS 6H 8S JS AC AS 8D 2H QS TS 3S AH 4H TH TD 3C 6S
8C 7D 4C 4S 7S 9H 7C 5D 2S KD 2D QH JD 6D 9D JC 2C KH 3D QC 6C 9S KC 7H 9C 5C
AC 2C 3C 4C 5C 6C 7C 8C 9C TC JC QC KC AD 2D 3D 4D 5D 6D 7D 8D TD 9D JD QD KD
AH 2H 3H 4H 5H 6H 7H 8H 9H KH 6S QH TH AS 2S 3S 4S 5S JH 7S 8S 9S TS JS QS KS
#
```

Sample Output

```
6 piles remaining: 40 8 1 1 1 1
1 pile remaining: 52
```

B - Problem B

Source file name: `boxes.cpp, boxes.py`
Time limit: c++: x seconds, python: x seconds

Today, besides SWERC'11, another important event is taking place in Spain which rivals it in importance: General Elections. Every single resident of the country aged 18 or over is asked to vote in order to choose representatives for the Congress of Deputies and the Senate. You do not need to worry that all judges will suddenly run away from their supervising duties, as voting is not compulsory.

The administration has a number of ballot boxes, those used in past elections. Unfortunately, the person in charge of the distribution of boxes among cities was dismissed a few months ago due to financial restraints. As a consequence, the assignment of boxes to cities and the lists of people that must vote in each of them is arguably not the best. Your task is to show how efficiently this task could have been done.

The only rule in the assignment of ballot boxes to cities is that every city must be assigned at least one box. Each person must vote in the box to which he/she has been previously assigned. Your goal is to obtain a distribution which minimizes the maximum number of people assigned to vote in one box.

In the first case of the sample input, two boxes go to the first city and the rest to the second, and exactly 100000 people are assigned to vote in each of the (huge!) boxes in the most efficient distribution. In the second case, 1, 2, 2 and 1 ballot boxes are assigned to the cities and 1700 people from the third city will be called to vote in each of the two boxes of their village, making these boxes the most crowded of all in the optimal assignment.

Input

The first line of each test case contains the integers N ($1 \leq N \leq 500\,000$), the number of cities, and B ($N \leq B \leq 2\,000\,000$), the number of ballot boxes. Each of the following N lines contains an integer a_i, ($1 \leq a_i \leq 5000000$), indicating the population of the i-th city.

A single blank line will be included after each case. The last line of the input will contain '-1 -1' and should not be processed.

The input must be read from standard input.

Output

For each case, your program should output a single integer, the maximum number of people assigned to one box in the most efficient assignment.

The output must be written to standard output.

Sample Input	Sample Output
2 7 200000 500000 4 6 120 2680 3400 200 -1 -1	100000 1700

C - Problem C

Source file name: `chronoscroll.cpp, chronoscroll.py`
Time limit: c++: 1 second, python: 1 second

Zlatan, Archmage-Engineer of the *Intergalactic Academy of AGRA*, was experimenting with a new device known as the *Chrono-Scroll Compiler*. Unfortunately, as often happens with experimental magic mixed with quantum mechanics, something went terribly wrong.

The Chrono-Scroll exploded.

What remained was a collection of *scroll fragments*, each fragment containing a sequence of enchanted pages. The internal order of the pages inside each fragment survived the explosion, but the links between fragments were completely lost.

Each page is labeled with a unique number from 1 to n. For every page, Zlatan's assistants managed to recover two pieces of information:

- which page originally came immediately before it,
- and which page originally came immediately after it.

If either piece of information is unknown, it is recorded as zero.

Zlatan knows that before the explosion:

- every page belonged to exactly one fragment,
- fragments were linear (no loops, no paradoxes), and
- together, the fragments formed a single enormous scroll.

To reconstruct the Chrono-Scroll, Zlatan allows only one operation: *the end of one fragment may be attached to the beginning of another fragment*, preserving the internal order of both fragments. Pages inside a fragment may not be reordered, rotated, reversed, or teleported through time.

However, Zlatan is extremely picky. Among all possible ways to reconstruct the full scroll, he demands the one whose sequence of page numbers is lexicographically smallest.

Input

The first line of input contains one integer specifying the number of test cases to follow. The first lineof each test case contains an integer n ($1 \leq n \leq 2 \times 10^5$), the number of pages.

Each of the next n lines of each test case contains two integers a_i and b_i:

- a_i is the page that originally came immediately before page i, or 0 if page i was the first page of its fragment.
- b_i is the page that originally came immediately after page i, or 0 if page i was the last page of its fragment.

It is guaranteed that the input describes several valid fragments whose union can be reconstructed into a single linear scroll.

The input must be read from standard input.

Output

For each test case print n lines. For each page i, output two integers:

- the page immediately before i in the reconstructed scroll (or 0 if none),
- the page immediately after i in the reconstructed scroll (or 0 if none).

The reconstruction must:

- include all pages exactly once,
- preserve the internal order of each fragment, and
- result in the lexicographically smallest possible sequence of page numbers.

There must be an empty line between two consecutive cases.

The output must be written to standard output.

Sample Input	**Sample Output**
2 4 0 2 1 0 0 4 3 0 7 4 7 5 0 0 0 6 1 0 2 0 4 1 0	0 2 1 3 2 4 3 0 4 7 5 6 0 5 6 1 3 2 2 4 1 0

D - Problem D

Source file name: `queue.cpp, queue.py`
Time limit: c++: x seconds, python: x seconds

Your government has finally solved the problem of universal health care! Now everyone, rich or poor, will finally have access to the same level of medical care. Hurrah!

There's one minor complication. All of the country's hospitals have been condensed down into one location, which can only take care of one person at a time. But don' t worry! There is also a plan in place for a fair, efficient computerized system to determine who will be admitted. You are in charge of programming this system.

Every citizen in the nation will be assigned a unique number, from 1 to P (where P is the current population). They will be put into a queue, with 1 in front of 2, 2 in front of 3, and so on. The hospital will process patients one by one, in order, from this queue. Once a citizen has been admitted, they will immediately move from the front of the queue to the back.

Of course, sometimes emergencies arise; if you've just been run over by a steamroller, you can't wait for half the country to get a routine checkup before you can be treated! So, for these (hopefully rare) occasions, an expedite command can be given to move one person to the front of the queue. Everyone else's relative order will remain unchanged.

Given the sequence of processing and expediting commands, output the order in which citizens will be admitted to the hospital.

Input

Input consists of at most ten test cases. Each test case starts with a line containing P, the population of your country ($1 \leq P \leq 1000000000$), and C, the number of commands to process ($1 \leq C \leq 1000$).

The next C lines each contain a command of the form `'N'`, indicating the next citizen is to be admitted, or `E` x, indicating that citizen x is to be expedited to the front of the queue.

The last test case is followed by a line containing two zeros.

The input must be read from standard input.

Output

For each test case print the serial of output. This is followed by one line of output for each `'N'` command, indicating which citizen should be processed next. Look at the output for sample input for details.

The output must be written to standard output.

Sample Input	**Sample Output**
3 6 N N E 1 N N N 10 2 N N 0 0	Case 1: 1 2 1 3 2 Case 2: 1 2

E - Problem E

Source file name: `zlatan.cpp, zlatan.py`
Time limit: c++: 1 second, python: 1 second

After the Great Campaign of Expansion, the AGRA Empire stood victorious across dozens of territories. Zlatan, Supreme Strategist of the Empire, must now decide which commanders are worthy of joining the Inner Circle.

Each commander led a sequence of missions during the campaign. Their performance was recorded in detail, but Zlatan believes that excellence cannot be measured using a single number. Instead, he orders the commanders to be ranked according to a strict hierarchical evaluation doctrine.

Your task is to implement this doctrine exactly as described. Each commander is identified by a unique name consisting only of uppercase letters. For every commander, the archive records the number of missions completed, their strategic score, the number of failures they suffered, and the difficulty levels of the missions in the exact order they were completed. Commanders are compared using the following principles, applied strictly in order.

- First, commanders are evaluated by their *effective campaign weight*. This value is obtained by summing the difficulty levels of all missions except the two easiest ones. If a commander completed fewer than three missions, this value is defined as zero.
- If commanders are tied, Zlatan evaluates their *adjusted strategic efficiency*, defined as the strategic score divided by one plus the number of failures. However, if the difficulty of the last mission is strictly smaller than the average difficulty of all missions, the commander is considered unstable and this value is divided by two for comparison purposes. All comparisons must be exact.
- If commanders are still tied, Zlatan examines their *adaptive streak*. This is defined as the length of the longest contiguous subsequence of missions such that the difficulty never decreases and increases at least once within the subsequence. A subsequence of constant difficulty does not count.
- If a tie remains, commanders are compared by their *volatility index*, defined as the number of times the mission difficulty strictly decreases between consecutive missions. A lower value is better.
- Finally, if all previous criteria fail to break the tie, commanders are ordered lexicographically by name.

The final ranking lists commanders from highest rank to lowest rank.

Input

The first line contains an integer N ($1 \leq N \leq 2 \times 10^5$), the number of commanders.

For each commander:

- A line containing the commander's name.
- A line containing three integers L, S, and F:
  - $1 \leq L \leq 2 \times 10^5$
  - $0 \leq S, F \leq 10^9$
- A line containing L integers representing mission difficulties ($1 \leq$ difficulty $\leq 10^9$).

The sum of all values of L does not exceed 3×10^5.

The input must be read from standard input.

Output

For each test case, print the names of the commanders, one per line, in descending rank order.

The output must be written to standard output.

Sample Input	Sample Output
3 ZLATAN 6 120 2 2 3 4 5 5 6 ARES 6 120 2 2 3 4 5 4 6 HERA 6 120 2 2 2 2 2 2 2 4 DELTA 3 10 0 5 5 5 ALPHA 3 10 0 5 5 5 CHARLIE 3 10 0 5 5 5 BRAVO 3 10 0 5 5 5	ZLATAN ARES HERA ALPHA BRAVO CHARLIE DELTA